# Granularity-Aware Co-Design: Choosing *What Kind* of Hardware to Build

> **Target:** Rocket + RoCC + Gemmini in Chipyard. Every result measured under Verilator; no commercial EDA.
> **Prior work:** open-source, already running end-to-end on two OpenHW cores.

## Overview

### Given an application, what hardware should you build for it? A fused instruction? A coprocessor? A full accelerator?

CHIA can build all three, but it never decides which. `examples/riscv_extensions` is handed a published spec to implement; `examples/esp_accel_loop` is handed "implement memcpy." Nothing derives a candidate from a real application, and nothing decides at what *granularity* to accelerate it.

We propose that missing decision stage. One Spike trace and one cycle model classify each hot region by size, route it to the cheapest hardware that fits, and hand it to CHIA's existing agent loops to build and verify.

Nothing in CHIA does this today: `chia/simulators/` holds only gem5 and champsim, and Spike is used solely for lockstep verification, never as a trace source for analysis.

## Methodology

Each size of hot region has a different hardware answer, and a different cost to beat:

| hot region | route | cost to beat |
|---|---|---|
| **1–2 instructions** | edit Rocket's decoder | ~1 cycle (stays in-pipeline) |
| **3+ instructions** | RoCC coprocessor | offload latency (we calibrate it) |
| **loop nest / kernel** | Gemmini accelerator | DMA + config + data marshalling |

> The loop around that table: **classify → agent writes Chisel → Spike lockstep and riscv-dv verify → Verilator measures → thresholds recalibrate.**

These thresholds are measured, not guessed. On CV32E40P a custom instruction retires in 1 cycle, while offloading over CV-X-IF cost 2. So fusing two instructions can never win — and we measured exactly that: zero speedup. We recalibrate the same way for RoCC.

Note that tiers 2 and 3 both attach over RoCC. That makes granularity the only variable between them, rather than confounding it with interface and toolchain.

**Why coarse and fine interact.** An accelerator's speedup is limited by the work left behind on the CPU — moving data in and out, configuring, tiling — not by the accelerator's own throughput. So offloading a kernel *creates* new fine-grained work, which may then deserve its own instruction. One trace answers both questions.

Gemmini lets us measure this directly. Its ISA offers two ways to run a large matmul: hand-written tiling loops, or a single CISC instruction whose hardware unroller exists, in its authors' words, because of "CPU and loop overheads." Running both isolates the residual. It also tests whether our classifier can derive a design decision Gemmini's authors already made by hand.

**Scope for three weeks:** the classifier for all three tiers, the first two routes end-to-end on Rocket, and one Gemmini case. We will not claim more.

## Expected Results

Our cycle model is derived from core manuals rather than fitted to results. It already lands within **0.25%** of Verilator across 14 measurements on two cores, and always over-predicts. Four discovered instructions were built in RTL, and every measured saving landed inside its predicted range.

The results we expect to matter most are the ones a naive loop would get wrong. Both are already measured. A correct CV-X-IF multiply-accumulate, ranked first under naive scoring, won **zero** cycles. A fusion also won **zero**, because removing instructions exposed a load-use stall that the longer schedule had been hiding. Our screening step therefore reports a *range* rather than a number, and we report how often it held.

**Risks.**

1. Our 0.25% came from a simple memory system. Rocket has caches and an MMU, so we compose our pipeline model with `gem5.py` / `champsim.py` rather than assume the figure transfers.
2. Recompiling reschedules code, and a baseline trace cannot predict the new schedule. This is precisely why we report ranges.
3. Gemmini's hardware unroller may already absorb most of the residual. If it does, that is a real result, measured the same way.

**Deliverables.** A CHIA fork (BSD-3, upstreamable) adding a discovery node and a granularity-aware screening node alongside gem5 and champsim, reusing `chia/chipyard`'s existing build, cosim and Verilator nodes; `examples/mouna_loop/`; the Rocket pipeline model; and a 4-page paper. Agent involvement is disclosed per CHIA's AI-assisted contributions policy.

## Cost Estimate

**~$500.** $250 LLM API (Chisel generation and repair, 10–15 candidates across three tiers), $150 GCP CPU (Chipyard builds, Spike traces, benchmark runs), $100 contingency for verification retries. No FPGA hours and no commercial EDA licences required.